

# Homework 2

Kun Yuan

Oct 23, 2025

## 1 Problems

**A1.** Let  $g = Wx$  with  $W \in \mathbb{R}^{m \times n}$ ,  $x \in \mathbb{R}^n$ , and  $f = L(g)$  where  $L: \mathbb{R}^m \rightarrow \mathbb{R}$ . By the chain rule:

$$\nabla_W f = (\nabla_g L(g)) x^\top \in \mathbb{R}^{m \times n}.$$

Equivalently, in component form:

$$\frac{\partial f}{\partial W_{ij}} = (\nabla_g L(g))_i x_j.$$

Now consider the case where the input  $x$  is a matrix  $X \in \mathbb{R}^{n \times d}$ , i.e.,  $G = WX$  and  $f = L(G)$ . Compute the gradient  $\frac{\partial f}{\partial W}$ .

*Hint: You may use the Gateaux derivative  $df(W)[\Delta W] = \frac{d}{dt} f(W + t\Delta W)|_{t=0}$  to assist your calculation.*

**A2.** Consider the following two-variable function:

$$f(x) = \frac{1}{2}(x_1^2 + 2x_2^2), \quad x = (x_1, x_2) \in \mathbb{R}^2.$$

- (a) Find the tightest (smallest possible) Lipschitz smoothness constant  $L$  and the tightest (largest possible) strong convexity constant  $\mu$  for this function. Then compute the condition number  $\kappa = L/\mu$ .
- (b) Let the initial point be  $x^{(0)} = (1, 1)$ . Apply the gradient descent update

$$x^{(k+1)} = x^{(k)} - \gamma \nabla f(x^{(k)})$$

to obtain the sequence  $x^{(1)}, x^{(2)}, \dots$ . Use the following three step sizes:

$$\gamma_1 = \frac{1}{4}, \quad \gamma_2 = \frac{1}{3}, \quad \gamma_3 = \frac{1}{2}.$$

For each step size, derive an explicit formula for the iteration sequence. Based on your results, discuss how the choice of step size affects convergence and convergence rate.

Consider another two-variable function:

$$g(x) = \frac{1}{2}(x_1^2 + 10x_2^2), \quad x = (x_1, x_2) \in \mathbb{R}^2.$$

- (c) Write down directly the tightest smoothness constant  $L$ , strong convexity constant  $\mu$ , and condition number  $\kappa$ .
- (d) Take the initial point  $x^{(0)} = (1, 1)$  and step size  $\gamma = \frac{1}{11}$ . Write the explicit formula for the gradient descent iteration sequence. Referring to the convergence result in (a) (b), briefly explain how the properties of the objective function (such as  $L$ ,  $\mu$ , and  $\kappa$ ) influence the convergence rate.

**A3.** Consider the following simplified three-layer linear neural network:

$$h_1 = W_1 x, \quad h_2 = W_2 h_1, \quad h_3 = W_3 h_2, \quad L = f(h_3),$$

where  $x \in \mathbb{R}^d$ ,  $h_1 \in \mathbb{R}^p$ ,  $h_2 \in \mathbb{R}^q$ ,  $h_3 \in \mathbb{R}^s$ , and  $W_1 \in \mathbb{R}^{p \times d}$ ,  $W_2 \in \mathbb{R}^{q \times p}$ ,  $W_3 \in \mathbb{R}^{s \times q}$ ,  $f: \mathbb{R}^s \rightarrow \mathbb{R}$ .

- (a) Based on the idea of forward and backward propagation, design a suitable computation method to compute the gradients

$$\frac{\partial L}{\partial W_3}, \quad \frac{\partial L}{\partial W_2}, \quad \frac{\partial L}{\partial W_1},$$

aiming to minimize computational cost. Describe in detail which quantities are computed directly, which are stored for later use, and which have already been computed and stored in the forward pass.

- (b) Compute the computational cost (in FLOPs) for both the forward propagation and the backward propagation.

*Note: Computational cost is measured in floating-point operations (FLOPs). For matrix multiplication  $A = BC$  with  $A \in \mathbb{R}^{m \times p}$ ,  $B \in \mathbb{R}^{m \times n}$ ,  $C \in \mathbb{R}^{n \times p}$ , the FLOP count is  $2mnp$ , even if one dimension is  $n = 1$ . The cost of evaluating  $f(\cdot)$  and its derivative  $f'(\cdot)$  is considered negligible.*

- (c) For an  $N$ -layer linear neural network:

$$h_0 = x, \quad h_i = W_i h_{i-1} \quad (i = 1, \dots, N), \quad L = f(h_N),$$

where  $x \in \mathbb{R}^{d_0}$ ,  $h_i \in \mathbb{R}^{d_i}$ ,  $W_i \in \mathbb{R}^{d_i \times d_{i-1}}$ , write down the computational cost of forward and backward propagation. Roughly estimate how many times more expensive the backward propagation is compared to the forward propagation.